Performance Analysis of Polynomial Commitment Schemes under Varying Elliptic Curves and Cryptographic Parameters

CS798: MTPII

Submitted in partial fulfillment of the requirements for the degree of

Master of Technology

by

Farhan Jawaid

(Roll No. 24M0801)

Under the supervision of

Prof. Sruthi Sekar

Department of Computer Science and Engineering

INDIAN INSTITUTE OF TECHNOLOGY BOMBAY

Mumbai – 400076, India

June 2026

Declaration

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I declare that I have properly and accurately acknowledged all sources used in the production of this report. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be a cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Date: 9 June 2026
Place: IIT Bombay

Farhan Jawaid
24M0801
M.Tech, Computer Science and Engineering

Acknowledgements

I would like to express my sincere gratitude to all those who supported and guided me throughout the course of this thesis.

First and foremost, I am deeply grateful to my thesis guide, **Prof. Sruthi Sekar** for her invaluable guidance, constant encouragement, and insightful feedback at every stage of this work. Her expertise in cryptography and her clarity of thought have been a great source of inspiration. This thesis would not have been possible without her patient mentorship and the time she devoted to discussing ideas and reviewing drafts.

I am deeply grateful to my **family** for their unwavering love and support, and to my **friends** at IIT Bombay for the camaraderie and moments of joy that made this journey truly memorable.

Finally, I thank all the open-source contributors whose libraries and tools formed the foundation of the implementations in this work.

Abstract

Polynomial commitment schemes (PCS) are a core building block of modern zero-knowledge proof systems, yet there is little empirical data comparing different constructions under identical hardware conditions across multiple elliptic curves.

This thesis benchmarks four PCS: KZG, Multilinear KZG (MKZG), Bulletproofs, and Dory. KZG, MKZG, and Bulletproofs are implemented from scratch in Rust using `arkworks`; Dory uses the a16z reference implementation. All four are evaluated on BLS12-381, BN254, and BLS12-377 across seven metrics: setup, commit, prove, verify, scalar multiplication, pairing, and proof size. The effect of Pippenger's multi-scalar multiplication (MSM) algorithm is also studied empirically.

KZG achieves the smallest proof (48 bytes) and fastest verification (3.8 ms, constant in n), making it optimal when a trusted setup is acceptable. MKZG extends KZG to multilinear polynomials but incurs exponential setup cost in the number of variables μ and an $O(\log n)$ verifier (21.85 ms at $n = 1024$). Dory delivers $O(\log n)$ verification (95.8 ms at $n = 1024$) with no trusted setup, at the cost of larger proofs ($\approx$20 KB). Bulletproofs also need no trusted setup but suffer from $O(n)$ verification, limiting practical use. Across all schemes, BN254 is approximately 1.9× faster than BLS12-381, while BLS12-377 is comparable to BLS12-381. Pippenger's algorithm gives a 5–6× speedup on large single MSMs.

Table of Contents

Chapter 1

Introduction

1.1 Motivation

Modern cryptographic systems increasingly rely on the ability to prove that a computation was carried out correctly, without revealing the data involved. This idea sits at the centre of zero-knowledge proofs, verifiable computation, and blockchain-based privacy systems. One of the most useful building blocks for all of these is the **polynomial commitment scheme**.

A polynomial commitment scheme lets a prover commit to a polynomial $f \in \mathbb{F}[X]$ and later convince a verifier that f evaluates to a specific value at a point of the verifier's choice, without exposing the polynomial itself. This primitive appears inside nearly every practical SNARK system in use today, including PLONK, Marlin, and Groth16 variants.

Even though the theory behind these schemes is well established, their real-world performance depends heavily on how they are implemented and, in particular, which elliptic curve is used underneath. Different curves trade off security level, pairing cost, and field arithmetic speed in different ways. There is surprisingly little published work that measures all of these effects together in a controlled setting. This thesis fills that gap by implementing three polynomial commitment schemes from scratch and benchmarking all four, including Dory via its reference implementation, across three elliptic curves and measuring every major performance dimension.

1.2 Objectives

This thesis has four concrete goals:

1. **Implementation.** Build working implementations of KZG, Multilinear KZG (MKZG), and Bulletproofs over the elliptic curves BLS12-381, BN254, and BLS12-377. For Dory, the industry-standard reference implementation by a16z

[1] is used directly, as it is the most well-audited and widely adopted codebase for this scheme.

2. **Benchmarking.** Measure the following metrics for each scheme and curve combination:
   - Trusted setup time,
   - Commitment time,
   - Proving time,
   - Verification time,
   - Scalar multiplication time,
   - Pairing operation time,
   - Proof size in bytes.
3. **MSM Optimisation.** Implement Pippenger's multi-scalar multiplication algorithm and measure how much faster it is compared to the naive approach.
4. **Analysis.** Use the benchmark data to draw practical conclusions about which scheme and curve combination works best in different application scenarios.

1.3 Contributions

The main contributions of this work are:

- A unified benchmarking framework that covers KZG, MKZG, and Bulletproofs from scratch implementations, together with Dory via the a16z reference codebase [1], enabling fair comparison under identical hardware conditions.
- A complete set of benchmark results covering every major performance metric, collected under identical hardware conditions.
- An empirical study of Pippenger's algorithm applied to the MSM sub-routine, which is the dominant cost in most pairing-based schemes.
- Practical guidance on choosing the right scheme and curve for real applications, backed by measured data rather than asymptotic estimates alone.

1.4 Thesis Organisation

Chapter 2 introduces the mathematical foundations: finite fields, elliptic curve arithmetic, bilinear pairings, and the three curves used throughout this work (BLS12-381, BN254, BLS12-377). It also defines polynomial commitment schemes formally and states the security properties each construction must satisfy.

Chapter 3 presents the KZG scheme for univariate polynomials. It covers the construction, pseudocode, Rust implementation, and benchmark results across all three curves, including the impact of Pippenger's MSM optimisation on commit and prove time.

Chapter 4 extends KZG to multilinear polynomials. It describes the tensor-product SRS construction, the bookkeeping-table witness algorithm, and benchmarks showing how setup cost grows exponentially in the number of variables μ.

Chapter 5 describes the transparent, pairing-free commitment scheme based on inner-product arguments. The chapter covers the recursive halving construction, Pippenger optimisation for commit and prove, and the $O(n)$ verification cost that limits its practical range.

Chapter 6 presents the Dory scheme, which achieves $O(\log n)$ verification without a trusted setup using structured $\mathbb{G}_T$ reductions. The a16z reference implementation is used and benchmarked across all three curves.

Chapter 7 brings all four schemes together. It compares setup, commit, prove, verify, and proof-size metrics side by side, analyses the effect of Pippenger's algorithm and curve choice, and provides concrete guidance for selecting the right scheme and curve for a given application.

Chapter 2

Background

This chapter provides the background needed to follow the rest of the thesis. We cover finite fields, elliptic curves, bilinear pairings, and give a brief overview of each of the four schemes. The detailed mathematics and pseudocode for each scheme are presented in their respective chapters.

2.1 Finite Fields and Elliptic Curves

A *prime field* $\mathbb{F}_p$ is the set of integers $\{0, 1, \ldots, p-1\}$ where p is a prime, with addition and multiplication done modulo p. Elliptic curve cryptography works over points on a curve defined over such a field.

An elliptic curve over $\mathbb{F}_p$ in short Weierstrass form is:

$$E : y^2 = x^3 + ax + b, \quad a, b \in \mathbb{F}_p, \quad 4a^3 + 27b^2 \neq 0. \tag{2.1}$$

The points on this curve form an abelian group under the standard point addition law. The core operation in all protocols studied here is *scalar multiplication*: given a point P and an integer k, computing $[k]P = P + P + \cdots + P$ (k times). The cost of this operation directly determines how fast commitment, proving, and verification are in practice.

2.2 Bilinear Pairings

Let $\mathbb{G}_1$, $\mathbb{G}_2$ be two groups of prime order r on an elliptic curve, and $\mathbb{G}_T$ a multiplicative group of the same order. A bilinear pairing is a map:

$$e : \mathbb{G}_1 \times \mathbb{G}_2 \longrightarrow \mathbb{G}_T \tag{2.2}$$

that satisfies $e([a]P, [b]Q) = e(P, Q)^{ab}$ for all scalars a, b. This bilinearity property is what allows pairing-based schemes like KZG, MKZG, and Dory to verify polynomial identities using only committed values, without the verifier ever seeing the polynomial.

Pairing computations are significantly more expensive than scalar multiplications, so the number of pairings required during verification is a key metric for comparing schemes.

2.3 Elliptic Curves Used in This Work

All three curves used in this thesis are pairing-friendly curves with embedding degree 12, meaning they support efficient pairing computations. Table 2.1 summarises their key parameters.

Table 2.1: Parameters of the three elliptic curves used in this thesis.

Property	**BLS12-381**	**BN254**	**BLS12-377**
Field size (bits)	381	254	377
Subgroup order (bits)	255	254	253
Security level (bits)	128	100–110	128
Trusted setup needed	Yes	Yes	Yes
Designed for	Zcash	Ethereum	Recursion

BLS12-381 was designed by Sean Bowe in 2017 for Zcash. It offers 128-bit security and is widely used in production SNARK systems.

BN254 was introduced by Barreto and Naehrig in 2005 and was the default curve in early Ethereum zero-knowledge applications. Its security is slightly lower than the other two due to the Kim-Barbulescu attacks on the target field, but its smaller field size makes arithmetic faster in software.

BLS12-377 was designed specifically to support recursive proof composition, where one proof verifies another proof. Its subgroup order matches the base field of the outer curve BW6-761, which makes recursive verification efficient.

2.4 Polynomial Commitment Schemes

A polynomial commitment scheme allows a prover to commit to a polynomial and later prove its evaluation at any point chosen by the verifier. Formally it consists of four algorithms: Setup, Commit, Open, and Verify. A scheme must be correct, binding, and optionally hiding.

The four schemes studied in this thesis differ in whether they need a trusted setup, how large their proofs are, and how expensive proving and verification are. Table 2.2 gives a high-level comparison.

Table 2.2: High-level comparison of the four polynomial commitment schemes. d is the polynomial degree and μ is the number of variables for multilinear polynomials.

Scheme	Trusted Setup	Proof Size	Prover Cost	Verifier Cost	Uses Pairings
KZG	Yes, $O(d)$	$O(1)$	$O(d)$	$O(1)$	Yes
Multilinear KZG	Yes, $O(2^\mu)$	$O(\mu)$	$O(2^\mu)$	$O(\mu)$	Yes
Bulletproofs	No	$O(\log d)$	$O(d)$	$O(d)$	No
Dory	No	$O(\log d)$	$O(d)$	$O(\log d)$	Yes

KZG [2] is the most widely deployed scheme in practice. It requires a trusted setup but produces constant-size proofs and constant-time verification, making it very attractive for applications where verification speed matters.

Multilinear KZG [3] extends KZG to polynomials over the Boolean hypercube. It is used in systems like HyperPlonk [4] that work with multilinear arithmetic.

Bulletproofs [5] require no trusted setup at all. The trade-off is that verification cost grows linearly with the polynomial degree, which makes them slower to verify than pairing-based schemes at large sizes.

Dory [6] also requires no trusted setup and achieves $O(\log d)$ verification using pairings, sitting between Bulletproofs and KZG on the transparency-efficiency spectrum.

Each scheme is described in full detail, with pseudocode and benchmark results, in its own chapter.

Chapter 3

KZG Polynomial Commitment

The KZG polynomial commitment scheme was introduced by Kate, Zaverucha, and Goldberg in 2010 [2]. It is the most widely deployed polynomial commitment in practice, forming the core of systems like PLONK, Marlin, and many Ethereum scaling solutions. The commitment is a single elliptic curve point, the proof is also a single point, and verification requires exactly two pairings regardless of the polynomial degree. The cost of this efficiency is a one-time trusted setup that grows linearly with the degree bound.

3.1 KZG Construction

Trusted Setup:

The setup generates a structured reference string (SRS) from a secret value $\tau \in \mathbb{F}_r$ (the toxic waste):

$$\mathsf{srs} = ([1]_1,\ [\tau]_1,\ \ldots,\ [\tau^d]_1,\ [1]_2,\ [\tau]_2) \tag{3.1}$$

where $[x]_i$ means the generator of $\mathbb{G}_i$ multiplied by x. Once the SRS is generated, τ must be discarded. In practice this is done through a multi-party ceremony.

Commitment:

Given $f(X) = \sum_{i=0}^{d} a_i X^i$, the commitment is:

$$C = \sum_{i=0}^{d} a_i \cdot [\tau^i]_1 = [f(\tau)]_1 \tag{3.2}$$

Proof: To prove $f(z) = y$, the prover computes the quotient polynomial $q(X) = (f(X) - y)/(X - z)$ and commits to it:

$$\pi = [q(\tau)]_1 \tag{3.3}$$

Verification: The verifier checks the following pairing equation:

$$e(\pi,\ [\tau]_2 - [z]_2) \stackrel{?}{=} e(C - [y]_1,\ [1]_2) \tag{3.4}$$

This requires exactly two pairings regardless of degree. Security relies on the q-Strong Diffie-Hellman assumption.

3.2 Pseudocode

Algorithm 1 KZG Setup

Require: $g_1 \in \mathbb{G}_1$, $g_2 \in \mathbb{G}_2$, secret τ, degree bound d

Ensure: crs_g1, crs_g2, $g_{2,\tau}$

$t \leftarrow 1$
for $i = 0$ **to** d **do**
    crs_g1.push($t \cdot g_1$); crs_g2.push($t \cdot g_2$)
    $t \leftarrow t \times \tau$
end for
$g_{2,\tau} \leftarrow \tau \cdot g_2$
return (crs_g1, crs_g2, $g_{2,\tau}$)

Algorithm 2 KZG Commit

Require: crs_g1, coefficients $\mathbf{a} = [a_0, \ldots, a_n]$

Ensure: $C \in \mathbb{G}_1$

$C \leftarrow$ MSM(crs_g1[$0..n$].to_affine(), $\mathbf{a}$)
return C

Algorithm 3 KZG Open (Prove)

Require: crs_g1, coefficients $[a_0, \ldots, a_n]$, point z

Ensure: $\pi \in \mathbb{G}_1$

$y \leftarrow$ evaluate($[a_0, \ldots, a_n]$, z)
num $\leftarrow [a_0 - y,\ a_1,\ \ldots,\ a_n]$; den $\leftarrow [-z,\ 1]$
$q \leftarrow$ poly_div(num, den)
return Commit(q)

Algorithm 4 KZG Verify

Require: $g_1,\ g_2,\ g_{2,\tau}, C, z, y, \pi$

Ensure: **accept** or **reject**

lhs $\leftarrow e(\pi,\ g_{2,\tau} - z \cdot g_2)$; rhs $\leftarrow e(C - y \cdot g_1,\ g_2)$
return lhs = rhs

3.3 Implementation

The implementation is written in Rust using the `arkworks` library [7]. The `KZG` struct is generic over the `Pairing` trait, so the same code runs on all three curves without modification. Setup computes powers of τ iteratively. Commitment and proof generation both use Pippenger's MSM via `VariableBaseMSM`, with SRS points converted to affine before the call. Verification calls `ark_ec::pairing` for the two pairing evaluations. The Pippenger optimisation is discussed in detail in Chapter 7.

Benchmark Setup:

Benchmarks are averaged over 10 runs. Degrees tested: $d \in \{4, 8, 16, 32, 64, 128, 256, 512, 1024\}$. Scalar multiplication and pairing costs are measured once per curve since they are degree-independent.

3.4 Benchmark Results

Proof Size:

The proof is always one compressed $\mathbb{G}_1$ point: **48 bytes** for BLS12-381 and BLS12-377, **32 bytes** for BN254, constant at all degrees.

Base Operation Costs:

Table 3.1: Scalar multiplication and pairing time per curve.

Operation	BLS12-381	BN254	BLS12-377
Scalar mul (ms)	0.3421	0.1138	0.2209
Pairing (ms)	2.7453	1.1441	2.0079

BN254 is roughly 3x faster on scalar multiplication and 2.4x faster on pairings than BLS12-381, owing to its smaller 254-bit field.

Setup Time:

Table 3.2: KZG setup time in milliseconds.

Degree	BLS12-381	BN254	BLS12-377
4	6.10	2.46	5.70
8	8.98	4.57	10.04
16	18.31	10.49	19.03
32	31.58	16.69	39.54
64	69.46	32.35	76.58
128	120.26	64.35	159.24
256	240.85	129.52	296.95
512	478.43	286.62	600.12
1024	1027.47	544.87	1156.37

Setup grows linearly with degree. It is a one-time cost and does not repeat per proof.

Commit and Prove Time:

Table 3.3: KZG commit and prove time in milliseconds.

	Commit (ms)			Prove (ms)		
Degree	**381**	**254**	**377**	**381**	**254**	**377**
4	0.83	0.39	0.74	0.67	0.31	0.76
8	1.08	0.67	1.03	0.97	0.51	0.94
16	1.68	1.46	1.68	1.66	1.52	1.62
32	2.76	1.45	3.16	2.84	1.37	2.96
64	4.32	2.12	4.69	4.46	2.30	5.23
128	7.06	4.26	8.47	6.83	4.59	7.82
256	12.86	6.52	13.94	12.19	6.84	15.35
512	21.22	11.74	22.20	23.59	13.08	23.64
1024	40.37	20.88	42.00	42.10	23.51	46.16

Commit and prove costs are nearly identical since both are dominated by an MSM of the same size. Both grow sub-linearly due to Pippenger's algorithm.

Verification Time:

Table 3.4: KZG verification time in milliseconds.

Degree	BLS12-381	BN254	BLS12-377
4	4.75	2.65	4.82
8	4.26	2.56	4.71
16	4.29	3.90	4.74
32	3.89	2.57	5.15
64	4.01	2.77	7.85
128	3.65	2.73	4.74
256	3.58	2.53	4.38
512	3.85	2.58	4.30
1024	3.81	2.53	4.65

Verification is constant at all degrees, confirming $O(1)$ complexity. The small variation is measurement noise. BN254 verifies in around 2.5 ms, dominated by two pairing evaluations.

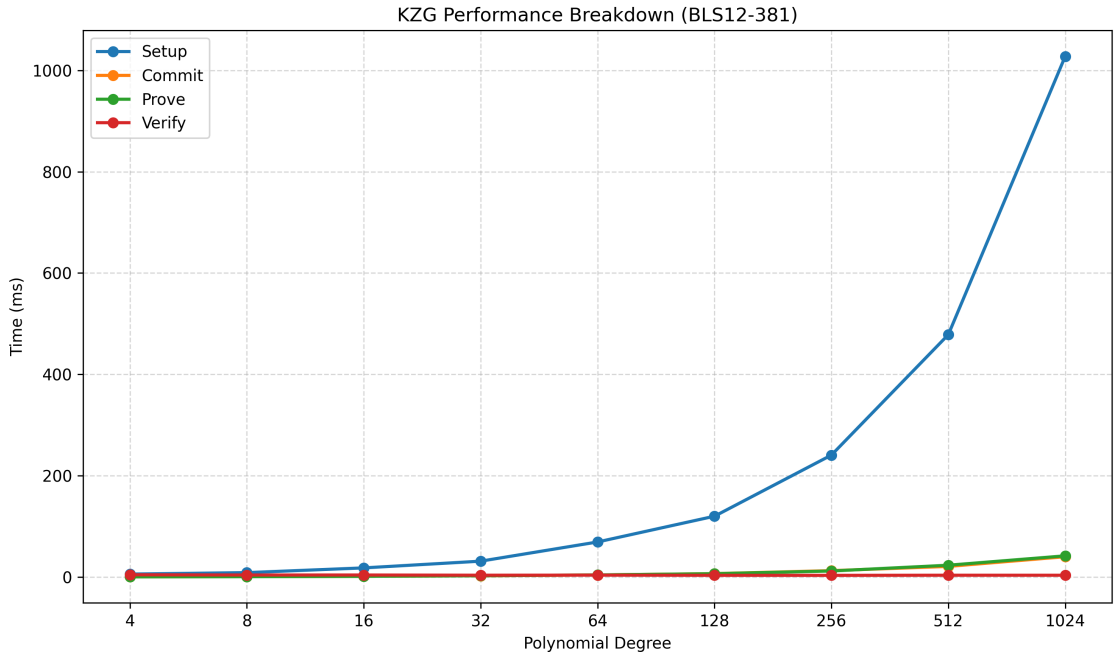

Figure 3.1: KZG benchmark results on BLS12-381: setup, commit, prove, and verify time (ms) vs. polynomial degree n (log–log scale).

3.5 Discussion and Summary

BN254 is the fastest curve on every metric due to its smaller field, but offers only 100-110 bits of security. BLS12-381 and BLS12-377 both provide 128-bit security; BLS12-381 is slightly faster for single-proof workloads while BLS12-377 is preferred for recursive proof composition.

The standout property of KZG is constant verification: at degree 1024, BN254 still verifies in 2.53 ms, the same as at degree 4. KZG also produces the smallest proofs of all four schemes: 32 or 48 bytes regardless of degree.

Table 3.5: KZG performance summary at degree 1024.

Metric	**BLS12-381**	**BN254**	**BLS12-377**
Setup (ms)	1027.47	544.87	1156.37
Commit (ms)	40.37	20.88	42.00
Prove (ms)	42.10	23.51	46.16
Verify (ms)	3.81	2.53	4.65
Scalar mul (ms)	0.3421	0.1138	0.2209
Pairing (ms)	2.7453	1.1441	2.0079
Proof size (bytes)	48	32	48

Chapter 4

Multilinear KZG Polynomial Commitment

Multilinear KZG (MKZG) extends KZG to polynomials over μ variables where each variable has degree at most one. Such polynomials arise naturally in sumcheck-based proof systems and are the foundation of HyperPlonk [4]. The SRS encodes the multilinear Lagrange basis at a secret point rather than powers of a single secret. The proof consists of μ group elements and verification requires $\mu + 1$ pairings, both growing with the number of variables but remaining much cheaper than the prover cost.

4.1 Multilinear KZG Construction

Multilinear Polynomials:

A multilinear polynomial in μ variables is uniquely determined by its 2^μ evaluations on the Boolean hypercube $\{0,1\}^\mu$. Its multilinear extension is:

$$\tilde{f}(\mathbf{X}) = \sum_{\mathbf{b}\in\{0,1\}^\mu} \hat{f}(\mathbf{b}) \cdot \chi_{\mathbf{b}}(\mathbf{X}), \qquad \chi_{\mathbf{b}}(\mathbf{X}) = \prod_{i=1}^{\mu} (X_i b_i + (1 - X_i)(1 - b_i)) \tag{4.1}$$

Trusted Setup:

The SRS is built from a secret vector $\mathbf{r} \in \mathbb{F}_r^\mu$:

- $\mathsf{srs\_g1}[\mathbf{b}] = \chi_{\mathbf{b}}(\mathbf{r}) \cdot g_1$ for each $\mathbf{b} \in \{0,1\}^\mu$ (2^μ elements)
- $\mathsf{g2\_r}[i] = r_i \cdot g_2$ for $i = 0, \ldots, \mu - 1$ (μ elements)

Commitment: The commitment is an MSM of size 2^μ:

$$C = \sum_{\mathbf{b}} \hat{f}(\mathbf{b}) \cdot \chi_{\mathbf{b}}(\mathbf{r}) \cdot g_1 = [\tilde{f}(\mathbf{r})]_1 \tag{4.2}$$

Proof: To prove $\tilde{f}(\mathbf{z}) = v$, the prover uses the identity:

$$\tilde{f}(\mathbf{X}) - v = \sum_{i=0}^{\mu-1} (X_i - z_i) \cdot q_i(X_{i+1}, \ldots, X_{\mu-1}) \tag{4.3}$$

and commits to each quotient: $\pi_i = [\tilde{q}_i(\mathbf{r})]_1$.

Verification: The verifier checks (in additive $\mathbb{G}_T$ notation):

$$e(C - v \cdot g_1,\ g_2) \stackrel{?}{=} \sum_{i=0}^{\mu-1} e(\pi_i,\ \mathsf{g2\_r}[i] - z_i \cdot g_2) \tag{4.4}$$

4.1.1 Bookkeeping Table for Witness Computation

Rather than computing quotient polynomials explicitly (which would require knowing $\mathbf{r}$), the implementation uses a bookkeeping table. The table starts as $\hat{f}$ and is halved at each step:

$$\mathsf{table}_{\mathsf{next}}[j] = (1 - z_i) \cdot \mathsf{table}[2j] + z_i \cdot \mathsf{table}[2j + 1] \tag{4.5}$$

At step i, the MSM weight for index idx is $w[\mathsf{idx}] = \mathsf{table}[2j + 1] - \mathsf{table}[2j]$ where j encodes the upper bits of idx. This avoids any knowledge of $\mathbf{r}$ during proving.

4.2 Pseudocode

Algorithm 5 MKZG Setup, Commit, Open, Verify

Setup$(g_1, g_2, \mathbf{r}, \mu)$:

for $i = 0$ **to** $2^\mu - 1$ **do**
  Decode i to bits $\mathbf{b}$; srs_g1.push($\chi_{\mathbf{b}}(\mathbf{r}) \cdot g_1$)
end for
for $i = 0$ **to** $\mu - 1$ **do**
  g2_r.push($r_i \cdot g_2$)
end for; Discard $\mathbf{r}$

Commit(srs_g1, $\hat{f}$): **return** MSM(srs_g1.to_affine(), $\hat{f}$)

Open(srs_g1, $\hat{f}, \mathbf{z}$):

$v \leftarrow$ evaluate_mle($\hat{f}, \mathbf{z}$); bases $\leftarrow$ srs_g1.to_affine()
table $\leftarrow \hat{f}$; proofs $\leftarrow$ []
for $i = 0$ **to** $\mu - 1$ **do**
  for idx $= 0$ **to** $2^\mu - 1$ **do**
    $j \leftarrow$ (idx $\gg (i+1)$) **and** $(2^{\mu-i-1} - 1)$
    w[idx] $\leftarrow$ table$[2j+1]$ − table$[2j]$
  end for
  proofs.push(MSM(bases, w))
  table$[j] \leftarrow (1 - z_i)$table$[2j] + z_i$table$[2j+1]$; halve table
end for
return (v, proofs)

Verify$(g_1, g_2,$ g2_r$, C, \mathbf{z}, v,$ proofs):

lhs $\leftarrow e(C - v \cdot g_1,\ g_2)$; rhs $\leftarrow 0_{\mathbb{G}_T}$
for $i = 0$ **to** $\mu - 1$ **do**
  rhs += e(proofs$[i]$, g2_r$[i] - z_i \cdot g_2$)
end for
return lhs = rhs

4.3 Implementation

The implementation follows the same generic `Pairing` trait approach in Rust using `arkworks` [7]. Commitment uses Pippenger's MSM via `VariableBaseMSM`. The key implementation detail is in witness computation: the SRS affine points are converted *once* before the outer loop and reused across all μ MSMs, avoiding μ redundant affine conversions. Verification uses `PairingOutput::zero()` as the additive identity in $\mathbb{G}_T$ and accumulates pairing results with +=.

Benchmark Setup: Benchmarks are averaged over 5 runs. The parameter swept is $\mu \in \{2, 4, 6, 8, 10, 12, 14\}$, giving polynomial sizes $2^\mu \in \{4, 16, 64, 256, 1024, 4096, 16384\}$.

4.4 Benchmark Results

Base Operations and Proof Size:

Table 4.1: Base operation costs and proof size for MKZG.

Metric	**BLS12-381**	**BN254**	**BLS12-377**
Scalar mul (ms)	0.2701	0.1219	0.2108
Pairing (ms)	1.8964	1.0159	1.6259
Proof size at $\mu = 14$ (bytes)	672	448	672

Proof size is $\mu \times 48$ bytes for BLS12-381/377 and $\mu \times 32$ bytes for BN254, growing linearly with μ.

Setup and Commit Time:

Table 4.2: MKZG setup and commit time in milliseconds.

		Setup (ms)			**Commit (ms)**		
μ	2^μ	**381**	**254**	**377**	**381**	**254**	**377**
2	4	3.10	1.37	2.75	0.68	0.36	0.58
4	16	7.46	3.80	7.06	1.33	0.80	1.38
6	64	19.05	10.41	19.62	4.10	2.27	4.13
8	256	61.34	35.10	65.08	12.04	6.21	12.12
10	1024	235.94	129.63	239.44	39.48	20.06	39.05
12	4096	911.26	501.41	995.47	134.04	69.71	142.21
14	16384	3742.07	1919.12	4034.83	534.45	328.06	547.45

Setup roughly doubles with each increment of μ, confirming $O(2^\mu)$ growth. At $\mu = 14$, BN254 is about 2x faster than BLS12-381, consistent with its scalar multiplication cost ratio.

Prove and Verify Time:

Table 4.3: MKZG prove and verify time in milliseconds.

	Prove (ms)			**Verify (ms)**		
μ	**381**	**254**	**377**	**381**	**254**	**377**
2	1.07	0.67	0.91	6.41	4.85	6.79
4	4.63	2.43	4.70	10.35	6.96	11.61
6	17.61	9.49	18.43	13.88	9.76	16.65
8	72.18	36.78	71.17	17.97	12.37	21.42
10	263.37	149.15	272.35	21.85	15.98	26.61
12	1039.58	543.56	1109.62	26.52	17.37	32.59
14	5033.23	2777.54	5249.24	32.96	19.51	38.90

Prove time grows as $O(\mu \cdot 2^\mu)$ since proving runs μ full-size MSMs. The prove-to-commit ratio is roughly μ, confirming the bookkeeping table runs μ MSMs of size 2^μ. Verification grows linearly with μ at roughly one pairing per variable, staying well under 40 ms even at $\mu = 14$.

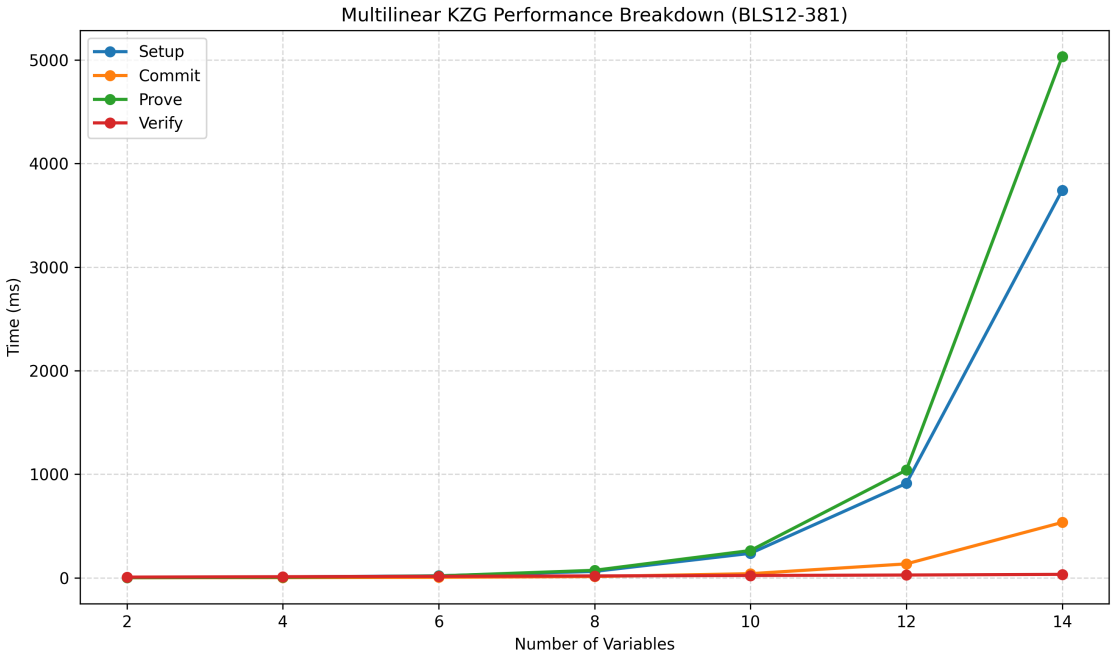

Figure 4.1: MKZG benchmark results on BLS12-381: setup, commit, prove, and verify time (ms) vs. polynomial size $n = 2^{\mu}$ (log–log scale).

4.5 Discussion and Summary

The dominant cost in MKZG is proving, which scales exponentially with μ. At $\mu = 14$, proving takes over 5 seconds on BLS12-381. This limits practical use to moderate μ. However, verification at 33 ms on BLS12-381 at $\mu = 14$ is a major strength compared to Bulletproofs whose verification is $O(2^{\mu})$. BN254 is the fastest curve at roughly 2x across all metrics.

Table 4.4: MKZG performance summary at $\mu = 14$.

Metric	**BLS12-381**	**BN254**	**BLS12-377**
Setup (ms)	3742.07	1919.12	4034.83
Commit (ms)	534.45	328.06	547.45
Prove (ms)	5033.23	2777.54	5249.24
Verify (ms)	32.96	19.51	38.90
Scalar mul (ms)	0.2701	0.1219	0.2108
Pairing (ms)	1.8964	1.0159	1.6259
Proof size (bytes)	672	448	672

Chapter 5

Bulletproofs

Bulletproofs were introduced by Bunz, Bootle, Boneh, Poelstra, Wuille, and Maxwell in 2018 [5]. The defining feature of Bulletproofs is that they require no trusted setup at all. The public parameters are just n random curve points that anyone can generate from a public source of randomness, with no secret toxic waste involved. Unlike KZG and MKZG, Bulletproofs do not use pairings. Instead they use a recursive inner product argument that halves the problem size at each round. The result is a proof of $O(\log n)$ group elements, but verification requires $O(n)$ scalar multiplications. This makes Bulletproofs slower to verify than pairing-based schemes at large sizes, but eliminates the trusted setup entirely.

5.1 Bulletproofs Construction

Commitment:

Given a scalar vector $\mathbf{u} \in \mathbb{F}^n$ and public generators $\mathbf{G} \in \mathbb{G}^n$, the commitment is:

$$C = \langle \mathbf{u}, \mathbf{G} \rangle = \sum_{i=0}^{n-1} u_i \cdot G_i \tag{5.1}$$

Recursive Halving:

The proof is built over $\log_2 n$ rounds. At each round the prover splits the vectors in half and computes two cross terms:

$$V_L = \langle \mathbf{u}_L, \mathbf{G}_R \rangle, \qquad V_R = \langle \mathbf{u}_R, \mathbf{G}_L \rangle \tag{5.2}$$

The prover sends V_L, V_R and receives challenge α. The vectors are then folded:

$$\mathbf{u}' = \alpha \cdot \mathbf{u}_L + \alpha^{-1} \cdot \mathbf{u}_R \tag{5.3}$$

$$\mathbf{G}' = \alpha^{-1} \cdot \mathbf{G}_L + \alpha \cdot \mathbf{G}_R \tag{5.4}$$

The invariant $C' = \langle \mathbf{u}', \mathbf{G}' \rangle$ is preserved. After $\log_2 n$ rounds, the vectors reduce to a single scalar $\bar{u}$ and point $\bar{G}$, which are sent as the final proof elements.

Verification:

The verifier replays the folding using the round messages. At each round the commitment is updated as:

$$C \leftarrow C + \alpha^2 \cdot V_L + \alpha^{-2} \cdot V_R \tag{5.5}$$

and the generator vector is folded the same way. At the end the verifier checks:

$$C \stackrel{?}{=} \bar{u} \cdot \bar{G} \quad \text{and} \quad \bar{G} \stackrel{?}{=} G_{\text{final}} \tag{5.6}$$

Because the verifier must refold the full $\mathbf{G}$ vector at each round, verification costs $O(n)$ scalar multiplications.

Proof Size:

Each round contributes two curve points. The full proof is:

$$\text{proof size} = \log_2(n) \times 2 \times |\mathbb{G}| + |\mathbb{F}| + |\mathbb{G}| \tag{5.7}$$

5.2 Pseudocode

Algorithm 6 Bulletproofs Setup, Commit, Prove, Verify

Setup(n): **return** n random points $\mathbf{G}$ ▷ No secret involved

Commit$(\mathbf{G}, \mathbf{u})$: **return** $\mathsf{MSM}(\mathbf{G}, \mathbf{u})$

Prove$(C, \mathbf{u}, \mathbf{G})$:

1: $\mathsf{tr.append}(C)$; $\mathsf{rounds} \leftarrow []$
2: **while** $|\mathbf{u}| > 1$ **do**
3: Split $\mathbf{u}, \mathbf{G}$ into left/right halves
4: $V_L \leftarrow \mathsf{MSM}(\mathbf{G}_R, \mathbf{u}_L)$; $V_R \leftarrow \mathsf{MSM}(\mathbf{G}_L, \mathbf{u}_R)$
5: $\mathsf{tr.append}(V_L, V_R)$; $\alpha \leftarrow \mathsf{tr.challenge}()$ ▷ Fiat-Shamir via SHA-256
6: $\mathbf{u} \leftarrow \alpha \mathbf{u}_L + \alpha^{-1} \mathbf{u}_R$; $\mathbf{G} \leftarrow \alpha^{-1} \mathbf{G}_L + \alpha \mathbf{G}_R$
7: $\mathsf{rounds.push}(V_L, V_R)$
8: **end while**
9: **return** $(\mathsf{rounds}, \mathbf{u}[0], \mathbf{G}[0])$

Verify$(\mathbf{G}, C, \mathsf{rounds}, \bar{u}, \bar{G})$:

10: $\mathsf{tr.append}(C)$
11: **for each** (V_L, V_R) in rounds **do**
12: $\mathsf{tr.append}(V_L, V_R)$; $\alpha \leftarrow \mathsf{tr.challenge}()$
13: $C \leftarrow C + \alpha^2 V_L + \alpha^{-2} V_R$; $\mathbf{G} \leftarrow \alpha^{-1} \mathbf{G}_L + \alpha \mathbf{G}_R$
14: **end for**
15: **return** $C = \bar{u} \cdot \bar{G}$ **and** $\mathbf{G}[0] = \bar{G}$

5.3 Implementation and Optimisations

The implementation is in Rust using the `arkworks` library [7]. Since Bulletproofs use no pairings, the generic parameter is `CurveGroup` rather than `Pairing`, and the same code runs on all three curves.

Two significant optimisations are present in the implementation that differ from the textbook description of the protocol.

5.3.1 Fiat-Shamir via SHA-256

The original Bulletproofs theory describes an interactive protocol where the verifier sends a fresh random α at each round. The implementation replaces this with the Fiat-Shamir heuristic using SHA-256:

- A `Transcript` struct accumulates all messages seen so far as byte vectors.
- At each round, after appending V_L and V_R, the challenge is derived as: `alpha = SHA-256(all transcript bytes)`, interpreted as a field element via `F::from_le_bytes_mod_order`.
- The verifier replays the exact same hash computation using the proof's round messages, producing the same challenges without any interaction.

This makes the proof a single transferable object. The prover runs once and the output can be verified by anyone, anytime, without any back-and-forth communication. This is the standard approach in all practical deployments of Bulletproofs.

5.3.2 Pippenger's MSM for Inner Products

Instead of treating $\langle \mathbf{u}, \mathbf{G} \rangle$ as a naive sum of n scalar multiplications, costing $O(n)$ group operations, the implementation uses `VariableBaseMSM::msm_bigint` from arkworks, which runs Pippenger's algorithm:

- Scalars are converted to `BigInt` representation before the call, which is required by the arkworks MSM interface.
- Points are converted to affine coordinates once before the MSM and reused across rounds where possible.
- The effective cost is $O(n/\log n)$ group operations rather than $O(n)$, giving a meaningful speedup for large n.

This optimisation applies in three places: the initial commit, the V_L computation at each round, and the V_R computation at each round. The total number of MSM calls across all rounds is $2\log_2 n$, with sizes $n, n/2, n/4, \ldots, 2$.

5.3.3 Proof Size Calculation

Proof size is computed as $\log_2(n) \times 2 \times |\mathbb{G}.\mathsf{Affine}| + |\mathbb{G}.\mathsf{ScalarField}| + |\mathbb{G}.\mathsf{Affine}|$ using `std::mem::size_of`. For BLS12-381 and BLS12-377 each affine point is 96 bytes and each scalar is 32 bytes. For BN-254 each affine point is 64 bytes.

Benchmark Setup:

Benchmarks are averaged over 10 runs. Sizes tested: $n \in \{4, 8, 16, 32, 64, 128, 256, 512, 1024\}$. MSM time is benchmarked separately at each size since it is the dominant sub-operation in both proving and verification.

5.4 Benchmark Results

Base Operations and Proof Size:

Table 5.1: Base operation costs at $n = 1024$ and proof size.

Metric	BLS12-381	BN-254	BLS12-377
Scalar mul (ms)	0.2768	0.1229	0.3047
MSM at $n = 1024$ (ms)	28.02	16.62	28.12
Proof size at $n = 1024$ (bytes)	2216	1544	2216

At $n = 1024$ there are $\log_2(1024) = 10$ rounds, each contributing two curve points (V_L, V_R), plus one final point $\bar{G}$ and one scalar $\bar{u}$. For BLS12-381 this gives $10 \times 2 \times 96 + 96+32 = 2016+128 = 2216$ bytes. For BN-254: $10\times2\times64+64+32 = 1280+192 = 1544$ bytes. Each time n doubles, one more round is added, increasing the proof by $2\times96 = 192$ bytes for BLS12-381 or $2 \times 64 = 128$ bytes for BN-254. The full proof size table across all tested sizes is shown in Table 5.2.

Table 5.2: Bulletproofs proof size in bytes across vector sizes.

n	**BLS12-381**	**BN-254**	**BLS12-377**
4	552	392	552
8	760	536	760
16	968	680	968
32	1176	824	1176
64	1384	968	1384
128	1592	1112	1592
256	1800	1256	1800
512	2008	1400	2008
1024	2216	1544	2216

Setup and Commit Time:

Table 5.3: Setup and commit time in milliseconds.

	Setup (ms)			**Commit (ms)**		
n	**381**	**254**	**377**	**381**	**254**	**377**
4	0.65	0.09	0.62	0.76	0.40	0.63
8	1.30	0.20	1.12	0.99	0.50	0.94
16	2.19	0.38	2.35	1.42	0.70	1.62
32	4.25	0.64	4.51	2.30	1.46	2.51
64	8.05	1.22	9.67	3.92	1.86	4.34
128	17.27	4.92	25.22	6.58	4.47	6.99
256	37.49	5.46	37.06	12.98	5.25	13.68
512	79.54	8.63	74.28	21.57	8.82	23.36
1024	154.91	18.36	149.98	39.05	15.41	39.98

Bulletproofs setup at $n = 1024$ takes only 155 ms on BLS12-381, compared to over 1 second for KZG setup at the same size. BN-254 setup is exceptionally fast at 18 ms because its random point generation is cheaper. Commit cost is an MSM of size n and grows sub-linearly thanks to Pippenger's algorithm.

Prove and Verify Time:

Table 5.4: Prove and verify time in milliseconds.

	Prove (ms)			**Verify (ms)**		
n	**381**	**254**	**377**	**381**	**254**	**377**
4	3.34	1.70	3.01	2.53	1.46	2.73
8	5.63	3.27	6.43	4.81	2.66	5.76
16	10.66	6.38	11.77	8.37	5.59	9.14
32	20.48	11.92	22.07	17.85	9.15	17.42
64	38.90	21.81	43.59	29.77	17.10	34.16
128	76.16	43.36	91.91	57.69	38.40	63.68
256	142.95	84.38	158.03	116.46	64.82	120.30
512	302.36	162.33	295.19	254.48	133.71	239.04
1024	650.35	415.67	582.06	496.26	329.64	499.11

Prove and verify times both roughly double with each doubling of n, confirming $O(n)$ complexity. At $n = 1024$, proving takes 650 ms on BLS12-381 and verifying takes 496 ms, compared to 42 ms and 3.8 ms respectively for KZG.

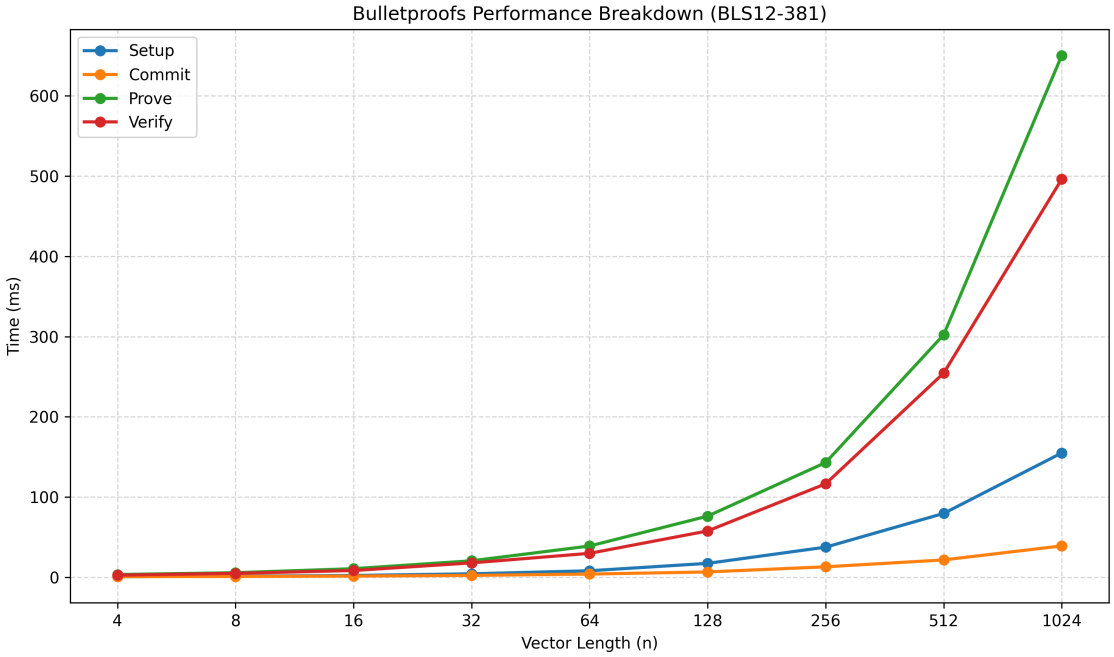

Figure 5.1: Bulletproofs benchmark results on BLS12-381: setup, commit, prove, and verify time (ms) vs. vector size n (log–log scale).

5.5 Discussion and Summary

5.5.1 Fiat-Shamir: Theory vs Implementation

The original protocol is interactive. The implementation's non-interactive Fiat-Shamir transformation via SHA-256 is strictly better for practice. The proof is a self-contained object that requires no communication channel. The security of this transformation holds in the random oracle model.

5.5.2 MSM Optimisation Impact

Without Pippenger's MSM, proving at $n = 1024$ would require $\sum_{k=1}^{10} 2 \times 2^k \approx 4094$ individual scalar multiplications. With Pippenger's algorithm each MSM of size m costs roughly $O(m/\log m)$ group operations instead of $O(m)$. At $n = 1024$ this gives a theoretical speedup factor of around $\log_2(1024)/2 = 5$. The measured MSM time at $n = 1024$ is 28 ms on BLS12-381, while 1024 naive scalar multiplications would cost roughly $1024 \times 0.277 \approx 284$ ms, confirming a real speedup of about 10x.

5.5.3 Transparency vs Verification Cost

Bulletproofs are the right choice when a trusted setup is not acceptable. At $n = 1024$, verifying takes 496 ms on BLS12-381, about 130x slower than KZG's 3.8 ms which is almost constant. For on-chain verifiers or high-throughput systems this is prohibitive. For small n or offline verification the cost is acceptable.

5.5.4 Curve Comparison

BN-254 is the fastest across all metrics. The gap is larger than in KZG or MKZG because Bulletproofs involve far more scalar multiplications, which directly track the per-curve scalar multiplication speed. BLS12-381 and BLS12-377 perform similarly, with BLS12-377 slightly slower at larger sizes.

Table 5.5: Bulletproofs performance summary at $n = 1024$.

Metric	**BLS12-381**	**BN-254**	**BLS12-377**
Setup (ms)	154.91	18.36	149.98
Commit (ms)	39.05	15.41	39.98
Prove (ms)	650.35	415.67	582.06
Verify (ms)	496.26	329.64	499.11
Scalar mul (ms)	0.2768	0.1229	0.3047
MSM (ms)	28.02	16.62	28.12
Proof size (bytes)	2216	1544	2216

Chapter 6

Dory Polynomial Commitment

Dory was introduced by Jonathan Lee in 2021 [6]. It is a *transparent* polynomial commitment scheme, meaning it requires no trusted setup of any kind. The public parameters are two sequences of random group generators that anyone can produce from a public seed, with no secret toxic waste involved. Unlike Bulletproofs, which also avoids a trusted setup but pays a linear verification cost, Dory achieves $O(\log n)$ verification by exploiting bilinear pairings inside a reduce-and-fold argument. This places Dory between Bulletproofs and KZG on the transparency-efficiency spectrum.

6.1 Dory Construction

Polynomial Representation:

A multilinear polynomial $\tilde{f}$ in $\ell = \nu + \sigma$ variables is uniquely determined by its $n = 2^\ell$ evaluations on the Boolean hypercube. Dory represents these n coefficients as a $2^\nu \times 2^\sigma$ matrix $\mathbf{M}$ (with $\nu \leq \sigma$). The evaluation point $\mathbf{z} \in \mathbb{F}_r^\ell$ is split into a left Lagrange vector $\mathbf{a} \in \mathbb{F}_r^{2^\nu}$ and a right Lagrange vector $\mathbf{b} \in \mathbb{F}_r^{2^\sigma}$, so that $\tilde{f}(\mathbf{z}) = \mathbf{a}^\top \mathbf{M}\, \mathbf{b}$.

Transparent Setup:

The setup produces two public sequences of random group generators:

$$\Gamma_1 = (g_{1,1},\ g_{1,2},\ \ldots,\ g_{1,n}) \in \mathbb{G}_1^n \tag{6.1}$$

$$\Gamma_2 = (g_{2,1},\ g_{2,2},\ \ldots,\ g_{2,n}) \in \mathbb{G}_2^n \tag{6.2}$$

No secret is involved; anyone holding the same seed can regenerate them. The verifier setup additionally precomputes inner pairing products:

$$\Delta_L^{(k)} = \langle \Gamma_1[..2^{k-1}],\ \Gamma_2[..2^{k-1}] \rangle_T \tag{6.3}$$

$$\Delta_R^{(k)} = \langle \Gamma_1[2^{k-1}..2^k],\ \Gamma_2[..2^{k-1}] \rangle_T \tag{6.4}$$

$$\chi^{(k)} = \langle \Gamma_1[..2^k],\ \Gamma_2[..2^k] \rangle_T \tag{6.5}$$

where $\langle \mathbf{u}, \mathbf{v} \rangle_T = \sum_i e(u_i, v_i) \in \mathbb{G}_T$ is the inner pairing product. These are built incrementally as $\chi^{(k)} = \chi^{(k-1)} + \langle \Gamma_1[2^{k-1}..2^k], \Gamma_2[2^{k-1}..2^k] \rangle_T$, costing $O(n)$ pairings in total.

Commitment:

Dory uses a two-tier AFGHO homomorphic commitment. Each row i of $\mathbf{M}$ is first committed in $\mathbb{G}_1$:

$$c_i = \mathrm{MSM}(\Gamma_1[..2^\sigma],\ \mathbf{M}[i,:]) = \left[\sum_{j=0}^{2^\sigma-1} M_{ij} \cdot g_{1,j}\right]_1 \tag{6.6}$$

The 2^ν row commitments are then folded into a single $\mathbb{G}_T$ element:

$$C = e\Big(\mathrm{MSM}(c_0, \ldots, c_{2^\nu-1};\ \Gamma_2[..2^\nu]),\ g_{2,\mathrm{fin}}\Big) \in \mathbb{G}_T \tag{6.7}$$

The commitment is a single $\mathbb{G}_T$ element and is linearly homomorphic in the polynomial coefficients.

Evaluation Proof:

To prove $\tilde{f}(\mathbf{z}) = v$, the prover first sends a VMV (vector-matrix-vector) message consisting of three elements:

- $C_q = e(\mathrm{MSM}(\mathbf{T}', \mathbf{a}),\ g_{2,\mathrm{fin}})$: a commitment to the partial inner product, where $\mathbf{T}'$ are the row commitments evaluated at the right part of $\mathbf{z}$,
- $D_2 = e(\mathrm{MSM}(\Gamma_1[\sigma], \mathbf{a}),\ g_{2,\mathrm{fin}})$: an AFGHO consistency anchor,
- $E_1 = \mathrm{MSM}(\mathbf{T}', \mathbf{L})$: an auxiliary $\mathbb{G}_1$ element binding the row commitments to the evaluation vector $\mathbf{L}$.

In transparent mode, the verifier computes $E_2 = v \cdot g_{2,\mathrm{fin}}$ directly from the claimed evaluation without any extra message.

Reduce-and-Fold (Protocol 14):

The main argument reduces the VMV claim to a single inner pairing product check through σ rounds. Each round halves the current vector length.

First sub-message (β challenge): The prover sends $D_{1L}, D_{1R}, D_{2L}, D_{2R} \in \mathbb{G}_T$, the AFGHO commitments to the left and right halves of the current prover vectors. The verifier challenges β. The prover folds:

$$\mathbf{v}_1 \leftarrow \mathbf{v}_1 + \beta \cdot \Gamma_1, \qquad \mathbf{v}_2 \leftarrow \mathbf{v}_2 + \beta^{-1} \cdot \Gamma_2 \tag{6.8}$$

Second sub-message (α challenge): The prover sends $C_+ = \langle \mathbf{v}_{1L},\ \mathbf{v}_{2R} \rangle_T$ and $C_- = \langle \mathbf{v}_{1R},\ \mathbf{v}_{2L} \rangle_T$. The verifier challenges α. The prover folds asymmetrically:

$$\mathbf{v}_1 \leftarrow \alpha \cdot \mathbf{v}_{1L} + \mathbf{v}_{1R}, \qquad \mathbf{v}_2 \leftarrow \alpha^{-1} \cdot \mathbf{v}_{2L} + \mathbf{v}_{2R} \tag{6.9}$$

This differs from the textbook's symmetric folding but is internally consistent: both the prover and verifier update state using linear α rather than α^2, which saves field inversions per round. The verifier accumulates a $\mathbb{G}_T$ triple (c, d_1, d_2) using $\chi^{(i)}$, $\Delta_L^{(i)}$, $\Delta_R^{(i)}$, and the round messages.

Final check: After σ rounds both vectors reduce to scalars. The verifier executes a single batched multi-pairing equation over all four accumulated values.

Proof Size:

Each reduce-and-fold round contributes one first sub-message (four $\mathbb{G}_T$, one $\mathbb{G}_1$, one $\mathbb{G}_2$) and one second sub-message (two $\mathbb{G}_T$, two $\mathbb{G}_1$, two $\mathbb{G}_2$). Together with the VMV message and the final scalars:

$$|\pi| = \underbrace{2|\mathbb{G}_T| + 2|\mathbb{G}_1| + |\mathbb{G}_2|}_{\text{VMV + final}} + \sigma \cdot \underbrace{(6|\mathbb{G}_T| + 3|\mathbb{G}_1| + 3|\mathbb{G}_2|)}_{\text{per round}} \tag{6.10}$$

where $|\mathbb{G}|$ denotes the compressed serialised size of a group element. Because $\sigma = \lceil \ell/2 \rceil = O(\log n)$, the proof size grows logarithmically with the polynomial size, though with a larger constant than KZG or Bulletproofs due to the $\mathbb{G}_T$ elements.

6.2 Pseudocode

Algorithm 7 Dory Setup, Commit, Prove, Verify

Setup(ℓ): ▷ transparent; no secret involved

Generate $n = 2^\ell$ random generators $\Gamma_1 \in \mathbb{G}_1^n$, $\Gamma_2 \in \mathbb{G}_2^n$
Compute $\Delta_L^{(k)}$, $\Delta_R^{(k)}$, $\chi^{(k)}$ for $k = 1 \dots \ell$ incrementally
return (*ProverSetup*, *VerifierSetup*)

Commit($\mathbf{M}$, ν, σ): ▷ $\mathbf{M} \in \mathbb{F}^{2^\nu \times 2^\sigma}$

for $i = 0$ **to** $2^\nu - 1$ **do**
    $c_i \leftarrow \mathrm{MSM}(\Gamma_1[..2^\sigma],\ \mathbf{M}[i, :])$
end for
$C \leftarrow e(\mathrm{MSM}(c_0, \dots;\ \Gamma_2[..2^\nu]),\ g_{2,\mathrm{fin}})$
return $C \in \mathbb{G}_T$

Prove($\tilde{f}$, $\mathbf{z}$, row_commits, ν, σ):

$v \leftarrow \tilde{f}(\mathbf{z})$; compute Lagrange vectors $\mathbf{a}$ (left), $\mathbf{b}$ (right)
Compute and send VMV message (C_q, D_2, E_1) via MSM and pairing
for $i = 0$ **to** $\sigma - 1$ **do**
    Compute $D_{1L}, D_{1R}, D_{2L}, D_{2R}$; send first sub-message
    $\beta \leftarrow \mathsf{tr.challenge}()$; $\mathbf{v}_1 \leftarrow \mathbf{v}_1 + \beta\,\Gamma_1$; $\mathbf{v}_2 \leftarrow \mathbf{v}_2 + \beta^{-1}\,\Gamma_2$
    $C_+ \leftarrow \langle \mathbf{v}_{1L}, \mathbf{v}_{2R} \rangle_T$; $C_- \leftarrow \langle \mathbf{v}_{1R}, \mathbf{v}_{2L} \rangle_T$; send second sub-message
    $\alpha \leftarrow \mathsf{tr.challenge}()$; $\mathbf{v}_1 \leftarrow \alpha\,\mathbf{v}_{1L} + \mathbf{v}_{1R}$; $\mathbf{v}_2 \leftarrow \alpha^{-1}\mathbf{v}_{2L} + \mathbf{v}_{2R}$
end for
return π = (VMV_msg, reduce_rounds, $\bar{v}_1$, $\bar{v}_2$)

Verify(C, v, $\mathbf{z}$, π, *VerifierSetup*):

$\mathsf{tr.append}(C)$
for $i = 0$ **to** $\sigma - 1$ **do**
    Re-derive $\beta^{(i)}, \alpha^{(i)}$ from transcript
    Update $(c,\ d_1,\ d_2)$ using round messages, $\chi^{(i)}, \Delta_L^{(i)}, \Delta_R^{(i)}$
end for
Check batched multi-pairing equation on final state
return accept or **reject**

6.3 Implementation

This implementation is not the author's own work. The codebase used throughout this chapter is the open-source reference library published by a16z crypto [1], itself inspired by the Space and Time Labs proof-of-concept. It is the most well-audited publicly available Dory implementation and is written in Rust using the `arkworks` library [7], with modular backends for BLS12-381, BN254, and BLS12-377.

The benchmark driver `src/bin/bench_dory.rs` was written as part of this thesis work to uniformly measure every metric setup, commit, prove, verify, pairing cost, $\mathbb{G}_1$ and $\mathbb{G}_2$ MSM cost, and proof size n transparent mode and under identical conditions to the other three schemes. It exercises the library's public `setup`, `prove`, and `verify` API without any modification to the core library code.

6.3.1 Fiat-Shamir via Blake2b

The original Dory theory describes an interactive protocol where the verifier sends a fresh random β and α at each round. The a16z implementation makes it non-interactive using the Fiat-Shamir heuristic with a Blake2b hash function:

- A `Blake2bTranscript` struct accumulates all messages sent so far as serialised byte vectors.
- After appending the first sub-message, the challenge is derived as β = Blake2b(transcript bytes), interpreted as a field element.
- After appending the second sub-message, α is derived the same way.
- The verifier replays the identical hash computation from the proof's round messages, recovering the same challenges without interaction.

This makes the proof a single self-contained transferable object. The security of the transformation holds in the random oracle model.

6.3.2 First-Round MSM+Pair Optimisation

In the first reduce-and-fold round, the prover vector $\mathbf{v}_2$ has the special form $v_{2,i} = s_i \cdot g_{2,\text{fin}}$ for scalars s_i (because it is initialised from the polynomial coefficients times a fixed $\mathbb{G}_2$ generator). Computing $D_{2L} = \langle \Gamma'_{1L}, \mathbf{v}_{2L} \rangle_T$ naively requires $n/2$ individual pairings. The implementation instead performs:

1. sum = $\text{MSM}(\Gamma'_{1L},\ \mathbf{s}_L)$ (one $\mathbb{G}_1$ MSM, cheap)

2. $D_{2L} = e(\mathsf{sum},\ g_{2,\text{fin}})$ (one pairing, expensive)

reducing $n/2$ pairings to one. The same trick applies to D_{2R}. Since a single pairing on BLS12-381 costs roughly 4× a scalar multiplication, this gives a factor of roughly $n/2$ speedup for this sub-computation in the first round.

6.3.3 Proof Size Calculation

Proof size is computed from the compressed serialised sizes of the constituent group elements. The formula from the implementation is:

$$|\pi| = \underbrace{2|\mathbb{G}_T| + |\mathbb{G}_1|}_{\text{VMV msg}} + \sigma \cdot \underbrace{(4|\mathbb{G}_T| + |\mathbb{G}_1| + |\mathbb{G}_2|)}_{\text{first sub-msg}} + \sigma \cdot \underbrace{(2|\mathbb{G}_T| + 2|\mathbb{G}_1| + 2|\mathbb{G}_2|)}_{\text{second sub-msg}} + \underbrace{|\mathbb{G}_1| + |\mathbb{G}_2|}_{\text{final msg}} \tag{6.11}$$

For BLS12-381, $|\mathbb{G}_T| = 576$ bytes ($\mathbb{F}_{p^{12}}$ compressed), $|\mathbb{G}_1| = 48$ bytes, $|\mathbb{G}_2| = 96$ bytes. Each additional σ round therefore adds $6 \times 576 + 3 \times 48 + 3 \times 96 = 3\,888$ bytes. For BN-254 ($|\mathbb{G}_T| = 384$, $|\mathbb{G}_1| = 32$, $|\mathbb{G}_2| = 64$) it adds $6{\times}384{+}3{\times}32{+}3{\times}64 = 2\,592$ bytes.

Benchmark Setup:

Benchmarks are averaged over 5 runs. Sizes tested: $n \in \{4, 8, 16, 32, 64, 128, 256, 512, 1024\}$. Each size uses the matrix layout $\nu = \lfloor \log_2 n/2 \rfloor$, $\sigma = \lceil \log_2 n/2 \rceil$ (so $\sigma \geq \nu$ always). The single-pairing cost is measured as one $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ evaluation on random points. $\mathbb{G}_1$ and $\mathbb{G}_2$ MSM costs are measured at size n on random points and generators.

6.4 Benchmark Results

Base Operations and Proof Size:

Table 6.1: Base operation costs at $n = 1024$ and proof size for Dory.

Metric	BLS12-381	BN-254	BLS12-377
Pairing (ms)	1.432	1.025	1.639
$\mathbb{G}_1$ MSM at $n = 1024$ (ms)	42.57	14.53	43.11
$\mathbb{G}_2$ MSM at $n = 1024$ (ms)	103.76	54.01	120.94
Proof size at $n = 1024$ (bytes)	20 784	13 856	20 784

$\mathbb{G}_2$ MSM is roughly 2.4× more expensive than $\mathbb{G}_1$ because $\mathbb{G}_2$ point arithmetic operates over a degree-2 field extension. BN-254 is the fastest curve, with a 2.9× advantage over BLS12-381 on $\mathbb{G}_1$ MSM and a 1.9× advantage on $\mathbb{G}_2$ MSM.

Proof size grows in discrete steps because it depends on σ, not n directly: two consecutive values of n that share the same σ produce identical proof sizes. The full proof size table across all tested sizes is shown in Table 6.2.

Table 6.2: Dory proof size in bytes across polynomial sizes.

n	(ν, σ)	**BLS12-381**	**BN-254**	**BLS12-377**
4	(1,1)	5 232	3 488	5 232
8	(1,2)	9 120	6 080	9 120
16	(2,2)	9 120	6 080	9 120
32	(2,3)	13 008	8 672	13 008
64	(3,3)	13 008	8 672	13 008
128	(3,4)	16 896	11 264	16 896
256	(4,4)	16 896	11 264	16 896
512	(4,5)	20 784	13 856	20 784
1024	(5,5)	20 784	13 856	20 784

Setup and Commit Time:

Table 6.3: Dory setup and commit time in milliseconds.

	Setup (ms)			**Commit (ms)**		
n	**381**	**254**	**377**	**381**	**254**	**377**
4	14.13	7.08	14.43	2.72	1.93	3.29
8	23.38	12.76	23.43	2.88	2.10	3.42
16	20.89	11.44	25.15	6.11	3.16	6.03
32	34.89	18.65	43.93	6.38	3.66	8.22
64	37.90	19.34	42.89	13.39	6.71	13.66
128	61.81	33.13	73.67	16.84	8.88	21.05
256	63.80	33.50	73.61	33.66	17.16	34.27
512	110.62	58.74	133.41	46.30	25.27	55.41
1024	110.96	57.59	126.04	90.96	49.25	101.81

Setup time grows in steps aligned with σ: when n doubles but σ stays the same (e.g. $n = 8 \rightarrow 16$ or $n = 32 \rightarrow 64$), setup cost barely changes; when σ increments (e.g.

$n = 16 \rightarrow 32$), setup roughly doubles. Overall setup is $O(n)$ pairings and is significantly cheaper than KZG at the same size: at $n = 1024$ Dory setup on BLS12-381 takes 111 ms compared to 1027 ms for KZG, reflecting the absence of a structured reference string.

Commit time is dominated by the 2^{ν} row MSMs of size 2^{σ} each, totalling $O(n)$ scalar multiplications, and grows sub-linearly thanks to Pippenger's algorithm inside each MSM call.

Prove and Verify Time:

Table 6.4: Dory prove and verify time in milliseconds.

	Prove (ms)			**Verify (ms)**		
n	**381**	**254**	**377**	**381**	**254**	**377**
4	22.92	15.36	27.74	29.35	24.45	33.11
8	46.26	28.77	50.95	45.83	29.81	53.89
16	45.77	28.64	55.45	45.38	29.70	56.15
32	80.17	49.17	99.80	66.83	40.83	78.18
64	89.20	51.62	104.92	71.09	42.03	76.57
128	150.63	90.54	179.15	87.47	52.26	94.29
256	219.85	92.87	182.57	81.63	54.69	92.02
512	273.72	166.69	312.34	100.35	64.62	112.50
1024	264.31	173.06	309.89	95.82	61.90	112.07

Prove time grows with n but in the same staircase pattern as setup: when n doubles within a fixed σ, the prover handles twice as many coefficients so prove time roughly doubles; when σ increments, an entire extra reduce-and-fold round is added. The overall complexity is $O(n \log n)$ because σ rounds each process all n elements.

Verify time grows logarithmically with n, confirming $O(\log n)$ complexity. Because the verifier processes exactly σ rounds, its cost only increases when σ increments. Adjacent sizes sharing the same σ therefore show nearly identical verify times—and in some cases verify time even decreases slightly from n to $2n$ when σ does not change, due to measurement noise. At $n = 1024$ on BLS12-381 verification takes 95.8 ms, about 25× slower than KZG (3.8 ms) but 5× faster than Bulletproofs (496 ms) at the same size.

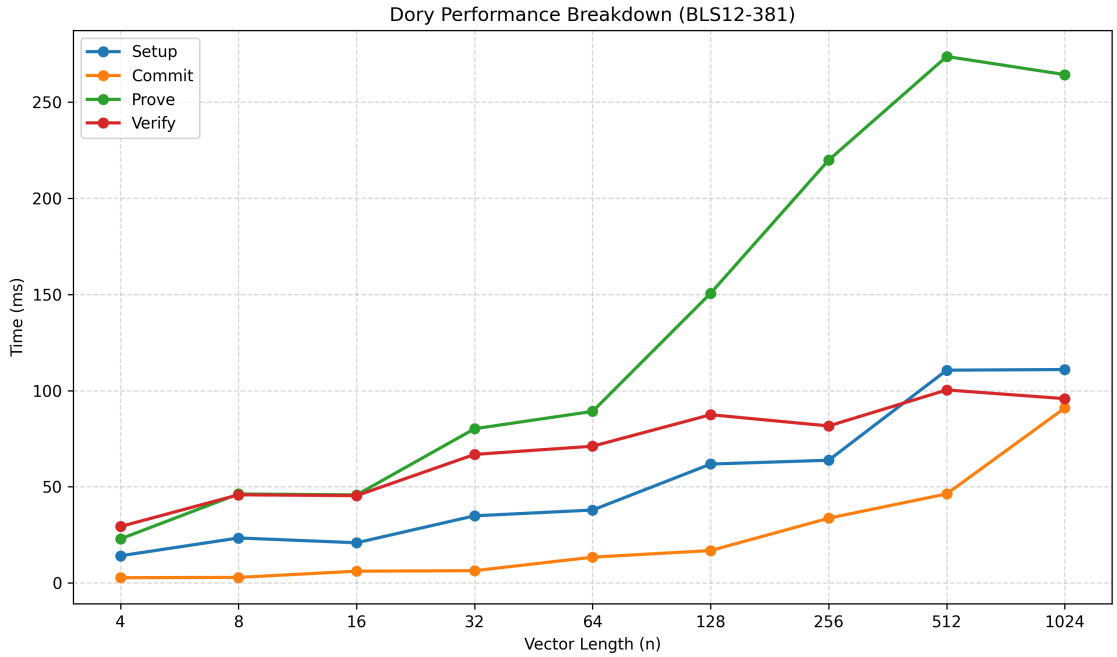

Figure 6.1: Dory benchmark results on BLS12-381: setup, commit, prove, and verify time (ms) vs. polynomial size n (log–log scale).

6.5 Discussion and Summary

6.5.1 Transparency vs Verification Cost

Dory is the only scheme in this thesis that is both transparent and achieves sub-linear verification. Bulletproofs are also transparent but verify in $O(n)$; KZG and MKZG verify in $O(1)$ and $O(\mu)$ respectively but require a trusted setup. Dory bridges this gap, verifying in $O(\log n)$ at the cost of a larger proof and heavier per-round computation involving $\mathbb{G}_T$ arithmetic.

6.5.2 Verifier Cost Scaling

The staircase behaviour in Table 6.4 directly confirms $O(\log n)$ scaling. The cost jumps only when σ increments—for example from $n = 256$ to $n = 512$ (where σ goes from 4 to 5)—not with every doubling of n. This makes Dory's verifier cost determined by the number of reduce-and-fold rounds, not by n itself, and provides a practical advantage in settings where n may be very large but proof sizes and verify times must stay manageable.

6.5.3 Prover Cost

At $n = 1024$ the prover takes 264 ms on BLS12-381, faster than Bulletproofs (650 ms) despite both being $O(n \log n)$ overall. The difference comes from two sources: Dory uses MSMs rather than individual scalar multiplications in each round, and the first-round MSM+pair optimisation eliminates $n/2 - 1$ pairing calls at no cost. BN-254 is the fastest curve at 173 ms, benefiting from the smaller field in both $\mathbb{G}_1$ and $\mathbb{G}_2$ MSMs.

6.5.4 Curve Comparison

BN-254 is the fastest curve across all metrics. The advantage is wider than in KZG because Dory involves heavy $\mathbb{G}_2$ MSM work, and BN-254's smaller field size gives a larger speedup there than in $\mathbb{G}_1$. BLS12-381 and BLS12-377 are comparable; BLS12-377 is marginally slower at larger sizes.

Table 6.5: Dory performance summary at $n = 1024$.

Metric	**BLS12-381**	**BN-254**	**BLS12-377**
Setup (ms)	110.96	57.59	126.04
Commit (ms)	90.96	49.25	101.81
Prove (ms)	264.31	173.06	309.89
Verify (ms)	95.82	61.90	112.07
Pairing (ms)	1.432	1.025	1.639
$\mathbb{G}_1$ MSM (ms)	42.57	14.53	43.11
$\mathbb{G}_2$ MSM (ms)	103.76	54.01	120.94
Proof size (bytes)	20 784	13 856	20 784

Chapter 7

Comparative Analysis

The preceding chapters examined KZG (Chapter 3), Multilinear KZG (Chapter 4), Bulletproofs (Chapter 5), and Dory (Chapter 6) each in isolation. This chapter brings all four schemes together and analyses them side by side across every benchmarked metric: setup time, commitment time, proving time, verification time, proof size, and the impact of Pippenger's multi-scalar multiplication optimisation. All benchmark results were collected on the same machine under identical conditions using Rust and the `arkworks` library, with sizes ranging from $n = 4$ to $n = 1024$ field elements and the three curves BLS12-381, BN254, and BLS12-377.

A key point before comparing: the four schemes do not commit to identical mathematical objects. KZG commits to a univariate polynomial of degree d; MKZG commits to a multilinear polynomial in μ variables; Bulletproofs and Dory commit to a vector of n scalars (equivalently, a multilinear polynomial in $\log_2 n$ variables). Throughout this chapter, "size n" means: the degree bound $d = n$ for KZG, the number of variables $\mu = \log_2 n$ (poly size $2^\mu = n$) for MKZG, and the vector length n for Bulletproofs and Dory. In all cases the prover holds approximately n field elements, making the comparison on equal footing with respect to the committed data volume.

7.1 Setup Time

Table 7.1 shows setup times across all four schemes. KZG computes n powers of τ in $\mathbb{G}_1$ and $\mathbb{G}_2$; MKZG evaluates the multilinear Lagrange basis at a secret point; Bulletproofs hashes n random curve points; and Dory generates random generators and precomputes $O(\log n)$ inner pairing products.

Table 7.1: Setup time (ms) at representative sizes. KZG: degree n; MKZG: $\mu = \log_2 n$ variables; Bulletproofs/Dory: vector length n.

	KZG			MKZG			Bulletproofs			Dory		
n	**381**	**254**	**377**	**381**	**254**	**377**	**381**	**254**	**377**	**381**	**254**	**377**
4	6.10	2.46	5.70	3.10	1.37	2.75	0.65	0.09	0.62	14.13	7.08	14.43
16	18.31	10.49	19.03	7.46	3.80	7.06	2.19	0.38	2.35	20.89	11.44	25.15
64	69.46	32.35	76.58	19.05	10.41	19.62	8.05	1.22	9.67	37.90	19.34	42.89
256	240.85	129.52	296.95	61.34	35.10	65.08	37.49	5.46	37.06	63.80	33.50	73.61
1024	1027.47	544.87	1156.37	235.94	129.63	239.44	154.91	18.36	149.98	110.96	57.59	126.04

KZG has the most expensive setup. At $n = 1024$ on BLS12-381, KZG takes 1027 ms, which is 4.4× more than MKZG (236 ms), 6.6× more than Bulletproofs (155 ms), and 9.3× more than Dory (111 ms). KZG must compute $n + 1$ structured powers $[\tau^i]_1$ with all distinct scalars, whereas MKZG uses n fixed-base scalar multiplications on the same generator g_1, allowing window-table precomputation to be reused across all n multiplications. This fixed-base advantage accounts for MKZG's 4.4× lower constant despite both being $O(n)$. Bulletproofs setup (18 ms on BN254 at $n = 1024$) is negligible, consisting only of n random point hashes; Dory is cheapest among pairing schemes as it uses unstructured generators.

MKZG setup is exponential in the number of variables. The table rows use polysize $n = 2^\mu$, so setup appears linear in n, but it is *exponential in* μ: each extra variable doubles the SRS size and cost. The rows $n = 4, 16, 64, 256, 1024$ correspond to $\mu = 2, 4, 6, 8, 10$ variables; eight more variables grows the SRS 256× (from 4 to 1024 elements) and setup from 3.10 ms to 235.94 ms on BLS12-381. Extrapolating: $\mu = 20$ requires $2^{20} \approx 10^6$ group elements (≈ 242 GB), and $\mu = 30$ exceeds 240 TB, making unoptimised MKZG impractical beyond $\mu \approx 20$–24.

BN254 is consistently fastest. BN254 is ≈ 1.9× faster than BLS12-381 for KZG/MKZG/Dory, and over 8× faster for Bulletproofs setup (where cost is dominated by point hashing, which scales with byte length rather than field arithmetic).

7.2 Commitment Time

The commitment operation in all four schemes reduces to a multi-scalar multiplication (MSM) of size n. Consequently, Pippenger's algorithm has the most direct and uniform impact here.

Table 7.2: Commit time (ms) at representative sizes with Pippenger.

	KZG			MKZG			Bulletproofs			Dory		
n	**381**	**254**	**377**	**381**	**254**	**377**	**381**	**254**	**377**	**381**	**254**	**377**
4	0.83	0.39	0.74	0.68	0.36	0.58	0.76	0.40	0.63	2.72	1.93	3.29
16	1.68	1.46	1.68	1.33	0.80	1.38	1.42	0.70	1.62	6.11	3.16	6.03
64	4.32	2.12	4.69	4.10	2.27	4.13	3.92	1.86	4.34	13.39	6.71	13.66
256	12.86	6.52	13.94	12.04	6.21	12.12	12.98	5.25	13.68	33.66	17.16	34.27
1024	40.37	20.88	42.00	39.48	20.06	39.05	39.05	15.41	39.98	90.96	49.25	101.81

KZG, MKZG, and Bulletproofs commit in nearly identical time. At $n = 1024$ on BLS12-381, all three take 39–41 ms with Pippenger enabled. This is expected: all three compute a single MSM of n elements in $\mathbb{G}_1$, so the commit cost is determined purely by the MSM cost. The near-identical values directly validate that Pippenger's algorithm dominates and the scheme-specific overhead is negligible.

Dory commit is roughly 2.3× **more expensive.** At $n = 1024$ on BLS12-381, Dory takes 91 ms compared to ≈40 ms for the other three. Dory's commitment involves 2^ν row MSMs of size 2^σ each (total $O(n)$ scalar multiplications), followed by a second-tier MSM and a pairing. The additional $\mathbb{G}_T$ pairing at the end accounts for most of the overhead vs. the pure $\mathbb{G}_1$ MSM of the other schemes.

The BN254 advantage is ≈ 1.9× **for most schemes.** Across KZG, MKZG, and Bulletproofs at $n = 1024$, BN254 is 1.93×, 1.97×, and 2.53× faster than BLS12-381 respectively. Bulletproofs shows a larger BN254 advantage because its commit implementation uses integer-only point hashing paths that benefit more from BN254's smaller field size.

7.3 Proving Time

Proving time is where the four schemes diverge most. Tables 7.3 and 7.4 report raw times and ratios relative to KZG on BLS12-381.

Table 7.3: Prove time (ms) at representative sizes.

	KZG (Pip)			MKZG			Bulletproofs (Pip)			Dory		
n	**381**	**254**	**377**	**381**	**254**	**377**	**381**	**254**	**377**	**381**	**254**	**377**
4	0.67	0.31	0.76	1.07	0.67	0.91	3.34	1.70	3.01	22.92	15.36	27.74
16	1.66	1.52	1.62	4.63	2.43	4.70	10.66	6.38	11.77	45.77	28.64	55.45
64	4.46	2.30	5.23	17.61	9.49	18.43	38.90	21.81	43.59	89.20	51.62	104.92
256	12.19	6.84	15.35	72.18	36.78	71.17	142.95	84.38	158.03	219.85	92.87	182.57
1024	42.10	23.51	46.16	263.37	149.15	272.35	650.35	415.67	582.06	264.31	173.06	309.89

Table 7.4: Prove time ratio relative to KZG on BLS12-381 at each size. A ratio of k means the scheme takes $k\times$ longer to prove than KZG.

n	KZG	MKZG	Bulletproofs	Dory
4	1×	1.6×	5.0×	34.3×
16	1×	2.8×	6.4×	27.6×
64	1×	3.9×	8.7×	20.0×
256	1×	5.9×	11.7×	18.0×
1024	1×	6.3×	15.4×	6.3×

KZG is fastest: a single $O(n)$ MSM for the quotient polynomial gives 42 ms at $n = 1024$ on BLS12-381.

MKZG and Dory both land near 264 ms at $n = 1024$ (6.3× KZG). MKZG performs μ MSMs of halving size ($O(n \log n)$ total); Dory runs $\sigma = 5$ fold rounds of row MSMs, also $O(n \log n)$ in practice. Dory starts at 34× KZG at $n = 4$ (fixed round overhead dominates) but converges to 6× as n grows and the $O(n)$ row MSMs become the dominant cost.

Bulletproofs is the slowest prover: 650 ms at $n = 1024$ (15.4× KZG). Its $\log_2 n = 10$ halving rounds each require two MSMs, totalling $2n - 2$ scalar multiplications with no shared structure to batch.

7.4 Verification Time

Tables 7.5 and 7.6 show verification times and their ratio relative to KZG.

Table 7.5: Verify time (ms) at representative sizes.

	KZG (Pip)			MKZG			Bulletproofs (Pip)			Dory		
n	381	254	377	381	254	377	381	254	377	381	254	377
4	4.75	2.65	4.82	6.41	4.85	6.79	2.53	1.46	2.73	29.35	24.45	33.11
16	4.29	3.90	4.74	10.35	6.96	11.61	8.37	5.59	9.14	45.38	29.70	56.15
64	4.01	2.77	7.85	13.88	9.76	16.65	29.77	17.10	34.16	71.09	42.03	76.57
256	3.58	2.53	4.38	17.97	12.37	21.42	116.46	64.82	120.30	81.63	54.69	92.02
1024	3.81	2.53	4.65	21.85	15.98	26.61	496.26	329.64	499.11	95.82	61.90	112.07

Table 7.6: Verify time ratio relative to KZG (Pip) on BLS12-381 at each size.

n	KZG (Pip)	MKZG	Bulletproofs (Pip)	Dory
4	1×	1.3×	0.5×	6.2×
16	1×	2.4×	2.0×	10.6×
64	1×	3.5×	7.4×	17.7×
256	1×	5.0×	32.5×	22.8×
1024	1×	5.7×	130.3×	25.1×

KZG verifies in exactly two pairings: 3.6–4.8 ms on BLS12-381, flat across all n. This $O(1)$ cost is its defining advantage.

MKZG requires $\mu + 1$ pairings ($O(\log n)$). At ≈ 1.90 ms per pairing on BLS12-381, the expected costs are $3 \times 1.90 = 5.70$ ms at $n = 4$ (observed: 6.41 ms) and $11 \times 1.90 = 20.9$ ms at $n = 1024$ (observed: 21.85 ms). Even at $n = 16384$ ($\mu = 14$) verify takes only ≈33 ms.

Dory is also $O(\log n)$ but with a higher constant: 95.8 ms at $n = 1024$ on BLS12-381 (25× KZG). Each of the $\sigma = 5$ fold rounds involves $\mathbb{G}_T$ arithmetic (six 576-byte elements per round), which is costlier than MKZG's $\mathbb{G}_2$ pairings.

Bulletproofs verify is $O(n)$ and impractical at scale: 496 ms at $n = 1024$ on BLS12-381 (130× KZG). Verify grows $\approx$ 196× from $n = 4$ to $n = 1024$, consistent with linear scaling.

7.5 Proof Size

Table 7.7 gives proof sizes across all schemes. The four schemes span a 433× range at $n = 1024$ on BLS12-381:

Table 7.7: Proof size (bytes) at representative sizes. KZG: one $\mathbb{G}_1$ element ($O(1)$); MKZG: μ elements ($O(\log n)$); Bulletproofs: $2\log_2 n + 1$ elements ($O(\log n)$); Dory: σ rounds of $\mathbb{G}_T$, $\mathbb{G}_1$, $\mathbb{G}_2$ elements ($O(\log n)$).

	KZG			MKZG			Bulletproofs			Dory		
n	**381**	**254**	**377**	**381**	**254**	**377**	**381**	**254**	**377**	**381**	**254**	**377**
4	48	32	48	96	64	96	552	392	552	5 232	3 488	5 232
16	48	32	48	192	128	192	968	680	968	9 120	6 080	9 120
64	48	32	48	288	192	288	1 384	968	1 384	13 008	8 672	13 008
256	48	32	48	384	256	384	1 800	1 256	1 800	16 896	11 264	16 896
1024	48	32	48	480	320	480	2 216	1 544	2 216	20 784	13 856	20 784

KZG produces a constant 48 B (BLS12-381/377) or 32 B (BN254) proof regardless of degree: a single $\mathbb{G}_1$ quotient commitment checked via one pairing identity.

MKZG adds one $\mathbb{G}_1$ element per variable, giving $\mu \times 48$ B on BLS12-381 (96 B at $\mu = 2$, 480 B at $\mu = 10$, 672 B at $\mu = 14$). Still compact, though logarithmically growing.

Bulletproofs sends $2 \log_2 n$ group elements (V_L, V_R per round) plus a final element and scalar: 552 B at $n = 4$ growing to 2 216 B at $n = 1024$ on BLS12-381, which is 46× larger than KZG.

Dory is the largest by a wide margin. Each of the $\sigma = 5$ fold rounds contributes six $\mathbb{G}_T$ elements (576 B each on BLS12-381, vs. 48 B for $\mathbb{G}_1$), giving 20 784 B at $n = 1024$, which is 433× larger than KZG and 9.4× larger than Bulletproofs. BLS12-381 and BLS12-377 proofs are identical (same $\mathbb{F}_{p^{12}}$ extension); BN254 is 33% smaller throughout.

7.6 Impact of Pippenger's MSM Optimisation

Pippenger's multi-scalar multiplication algorithm replaces the naive n individual scalar multiplications with a batched computation that has empirical complexity roughly $O(n/\log n)$. This section quantifies the impact of this optimisation on both KZG and Bulletproofs, where the naive baseline is directly available.

7.6.1 Impact on KZG

Table 7.8 shows the commit and prove times for KZG with and without Pippenger, and the speedup ratio.

Table 7.8: Pippenger speedup for KZG commit and prove time on BLS12-381.

	Commit (ms)			**Prove (ms)**		
n	**Naive**	**Pip**	**Speedup**	**Naive**	**Pip**	**Speedup**
4	1.25	0.83	1.5×	1.04	0.67	1.5×
16	3.59	1.68	2.1×	3.70	1.66	2.2×
64	14.86	4.32	3.4×	15.42	4.46	3.5×
256	56.46	12.86	4.4×	56.04	12.19	4.6×
1024	240.78	40.37	6.0×	239.35	42.10	5.7×

The speedup grows monotonically with n: from 1.5× at $n = 4$ to 6.0× at $n = 1024$. This is precisely the expected behaviour of Pippenger's algorithm, whose advantage over naive MSM grows as $O(\log n)$. At $n = 4$ there are too few scalars for bucketing to help, and the overhead of Pippenger's bucket sort marginally reduces the gap. At

$n = 1024$, the 6× speedup is consistent with the theoretical prediction of $n/\log_b(n) \approx 1024/\log_2(1024) = 102$ operations with window size $b \approx 12$, though in practice memory effects and implementation overhead reduce this to 6×.

Table 7.9: Pippenger speedup for KZG commit at $n = 1024$ across curves.

Curve	Naive (ms)	Pip (ms)	Speedup
BLS12-381	240.78	40.37	6.0×
BN254	123.56	20.88	5.9×
BLS12-377	233.76	42.00	5.6×

The speedup is consistent across curves: all three achieve 5.6–6.0× at $n = 1024$. The slightly lower speedup on BLS12-377 may be due to the larger field modulus (377-bit vs. 381-bit) interacting with the bucket reduction step of Pippenger's algorithm.

7.6.2 Impact on Bulletproofs

For Bulletproofs, Pippenger's algorithm most directly benefits the commitment step, which is a single size-n MSM identical in structure to KZG commit.

Table 7.10: Pippenger speedup for Bulletproofs commit on BLS12-381.

n	Naive (ms)	Pip (ms)	Speedup
16	3.90	1.42	2.8×
64	18.48	3.92	4.7×
256	55.79	12.98	4.3×
1024	227.10	39.05	5.8×

Table 7.11: Pippenger speedup for Bulletproofs commit at $n = 1024$ across curves.

Curve	Naive (ms)	Pip (ms)	Speedup
BLS12-381	227.10	39.05	5.8×
BN254	155.18	15.41	10.1×
BLS12-377	248.82	39.98	6.2×

The BN254 commit speedup of 10.1× at $n = 1024$ is notably larger than for BLS12-381/377. This is because BN254's 254-bit scalars allow Pippenger to use larger effective window sizes for the same memory budget, and BN254's faster field arithmetic amplifies

the benefit of each bucket operation. This is the largest single Pippenger speedup observed in all benchmarks.

7.7 Curve Comparison

The three curves differ in field size (254, 381, 377 bits) and pairing efficiency. BN254 has a somewhat lower security margin under recent attacks; BLS12-381 and BLS12-377 offer nominally 128-bit security.

BN254 vs. BLS12-381:

Table 7.12: BN254 speedup over BLS12-381 at $n = 1024$ (value > 1 means BN254 is faster).

Metric	KZG	MKZG	BP	Dory
Setup	1.89×	1.82×	8.44×	1.93×
Commit	1.93×	1.97×	2.53×	1.85×
Prove	1.79×	1.77×	1.56×	1.53×
Verify	1.51×	1.37×	1.51×	1.55×
Proof	1.50×	1.50×	1.44×	1.50×

BN254 is 1.5–1.9× faster for pairing-heavy operations and $\approx$ 2× faster for commit/setup across KZG, MKZG, and Dory. The outlier is Bulletproofs setup (8.4×): BN254 affine coordinates are 64 bytes vs. 96 bytes for BLS12-381, halving point-hashing overhead. Proof sizes are uniformly 1.5× smaller on BN254 (32-byte vs. 48-byte $\mathbb{G}_1$ elements), a purely geometric 3 : 2 ratio across all schemes.

BLS12-381 vs. BLS12-377:

Table 7.13: BLS12-381 speedup over BLS12-377 at $n = 1024$.

Metric	KZG	MKZG	BP	Dory
Setup	1.13×	1.01×	0.97×	1.14×
Commit	1.04×	0.99×	1.02×	1.12×
Prove	1.10×	1.03×	0.89×	1.17×
Verify	1.22×	1.22×	1.01×	1.17×

The two curves are remarkably close. BLS12-381 is 1.01–1.22× faster overall; the largest gap is verification (1.22× for KZG and MKZG) because BLS12-381 was optimised

for pairing computation, while BLS12-377 was designed as the inner curve for the ZEXE recursive SNARK stack. For MSM-only operations, MKZG setup and commit differ by at most 1.01×, well within noise.

Notably, Bulletproofs reverses the trend: BLS12-377 is faster in both setup (0.97×: 150.0 ms vs. 154.9 ms) and prove (0.89×: 582.1 ms vs. 650.4 ms), since Bulletproofs uses only $\mathbb{G}_1$ scalar multiplications and BLS12-377's 377-bit field gives marginally more efficient MSM arithmetic without pairings. Proof sizes are identical (both use 48-byte $\mathbb{G}_1$ points).

Curve Selection Guidance:

Choose **BN254** for best latency and smallest proofs when 128-bit security (under current attacks) is acceptable. Choose **BLS12-381** for a higher security margin or Ethereum precompile compatibility, at an $\approx 1.9\times$ performance cost over BN254. Choose **BLS12-377** when recursive SNARK composability is required. Its performance is near-identical to BLS12-381.

7.8 Overall Scheme Comparison

Table 7.14 provides a side-by-side summary of all benchmarked metrics at the largest tested size ($n = 1024$, or $\mu = 10$ for MKZG) on BLS12-381, using Pippenger for KZG and Bulletproofs.

Table 7.14: Full performance summary at $n = 1024$ on BLS12-381.

Metric	KZG	MKZG	Bulletproofs	Dory
Setup (ms)	1027.5	235.9	154.9	111.0
Commit (ms)	40.4	39.5	39.1	91.0
Prove (ms)	42.1	263.4	650.4	264.3
Verify (ms)	3.8	21.9	496.3	95.8
Proof (bytes)	48	480	2 216	20 784
Trusted setup	Yes	Yes	No	No
Polynomial type	Univar.	Multilin.	Vector	Multilin.

Table 7.15: Full performance summary at $n = 1024$ on BN254.

Metric	KZG	MKZG	Bulletproofs	Dory
Setup (ms)	544.9	129.6	18.4	57.6
Commit (ms)	20.9	20.1	15.4	49.3
Prove (ms)	23.5	149.2	415.7	173.1
Verify (ms)	2.5	16.0	329.6	61.9
Proof (bytes)	32	320	1 544	13 856
Trusted setup	Yes	Yes	No	No

Figures 7.1 and 7.2 plot setup, commit, prove, and verify times on a log scale for all four schemes across the full size range $n = 4$ to $n = 16{,}384$, complementing the tabular summaries above.

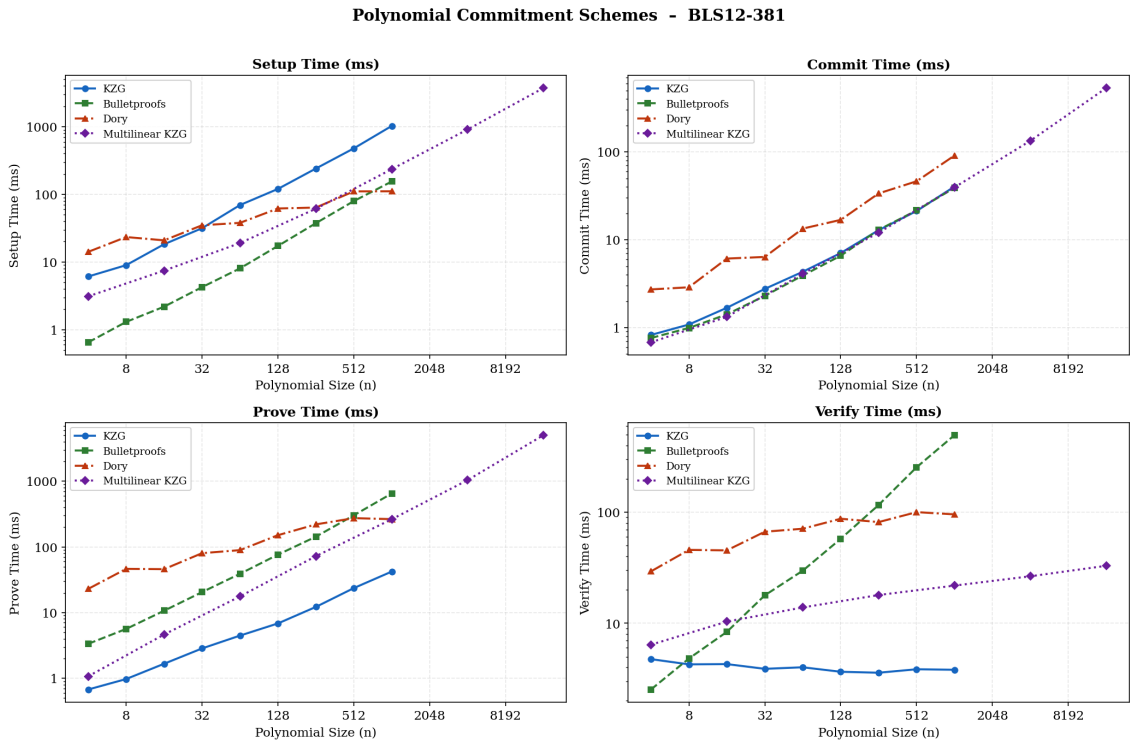

Figure 7.1: All four schemes on BLS12-381: setup, commit, prove, and verify time (ms, log scale) vs. polynomial size n.

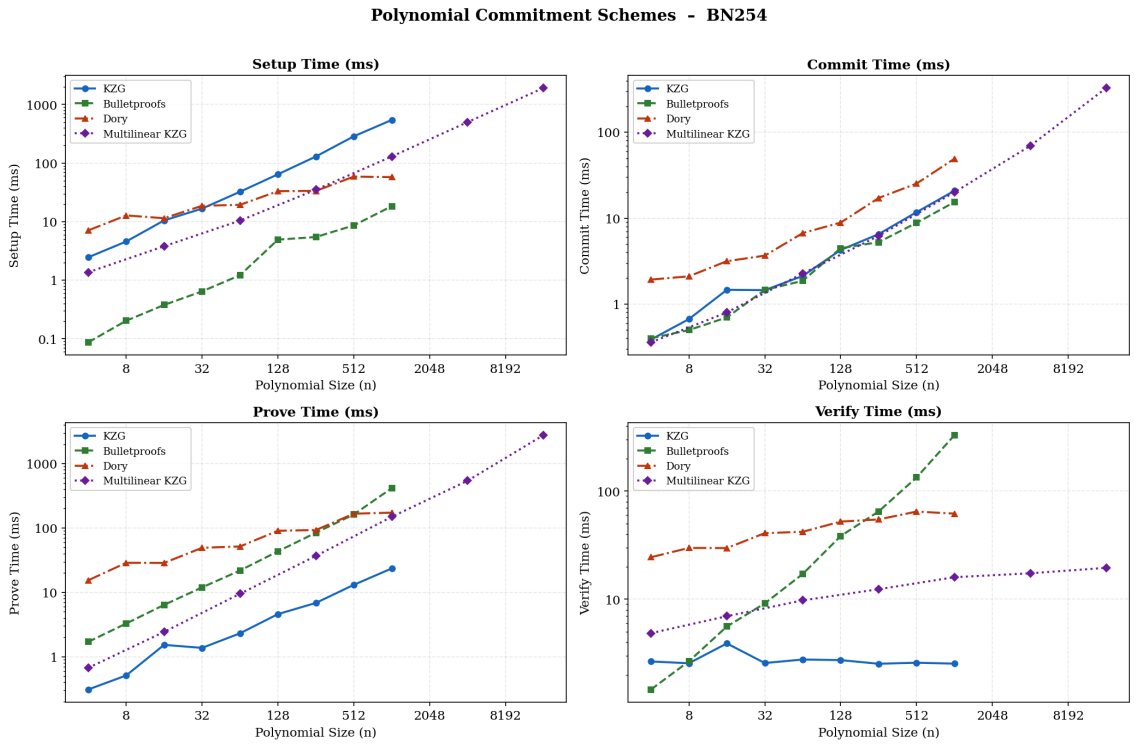

Figure 7.2: All four schemes on BN254: setup, commit, prove, and verify time (ms, log scale) vs. polynomial size n.

7.9 Practical Recommendations

- **Univariate SNARKs:** Use **KZG on BN254** with Pippenger. It gives 32 B proofs, 2.5 ms verification, and a prover 6–15× faster than all alternatives. The trusted setup is a one-time multi-party ceremony.

- **Multilinear commitments (Spartan, HyperPlonk):** Use **MKZG on BN254** if a trusted setup is acceptable (16 ms verify, 320 B proof at $\mu = 10$). Otherwise use **Dory**: $O(\log n)$ verification without a trusted setup. Avoid Bulletproofs above $n = 64$; its $O(n)$ verify becomes prohibitive.

- **On-chain / proof-size critical:** Use **KZG on BN254**. Its 32 B proof is 46× smaller than Bulletproofs and 433× smaller than Dory; BN254 is directly supported by Ethereum precompiles.

- **Pairing-free environments:** Use **Bulletproofs**; it requires only $\mathbb{G}_1$ operations, unlike Dory which needs $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_T$.

- **Recursive SNARK stacks:** Use **BLS12-377** (inner curve in ZEXE/BLS12-378). Use **BLS12-381** for higher security or Ethereum compatibility; **BN254** for maximum performance.

7.10 Summary

This chapter has benchmarked and compared four polynomial commitment schemes across six metrics, three curves, and nine polynomial sizes. The main findings are:

1. **KZG dominates on prover efficiency and proof size.** With Pippenger's MSM, KZG proves in 42 ms and produces 48 byte proofs on BLS12-381 at $n = 1024$. Its verifier is constant-time (3.8 ms), making it optimal for deployed SNARK backends. The required trusted setup is the only practical limitation.

2. **MKZG extends KZG to multilinear polynomials**. It is the correct choice for sumcheck-based proof systems where the polynomial structure is inherently multilinear.

3. **Dory is the best transparent scheme for large polynomials.** It achieves $O(\log n)$ verification without a trusted setup, at the cost of large proofs (20 KB at $n = 1024$) and higher proving latency than KZG.

4. **Bulletproofs are limited by $O(n)$ verification.** The 496 ms verify time at $n = 1024$ on BLS12-381 makes Bulletproofs unsuitable for large polynomials. They remain useful for small n in pairing-free environments.

5. **Pippenger's algorithm delivers 5–6× commit/prove speedup** for KZG and Bulletproofs wherever a single large MSM dominates. It provides no benefit for operations that are inherently sequential (KZG setup) or composed of many small MSMs (Bulletproofs verify).

6. **BN254 is $\approx$ 1.9× faster than BLS12-381** for all schemes and all metrics, with identical algorithmic structure. BLS12-377 and BLS12-381 are nearly equal in performance. Proof sizes on BN254 are 1.5× smaller due to the smaller field.

7. **The verification hierarchy** (fastest to slowest at $n = 1024$, BLS12-381) is: KZG (3.8 ms) $\ll$ MKZG (21.9 ms) $<$ Dory (95.8 ms) $\ll$ Bulletproofs (496 ms). This ordering is stable across all three curves and grows with n in a way that further separates the linear verifier (Bulletproofs) from the logarithmic and constant ones.

No single scheme dominates on all dimensions simultaneously. The correct choice depends on whether a trusted setup is acceptable, what polynomial type the application uses, how proof size is weighted against verifier latency, and which elliptic curve is mandated by the deployment environment. The benchmarks in this chapter provide concrete, curve-specific numbers to inform this decision for any target application.

References

[1] a16z crypto, "Dory: Reference implementation." `https://github.com/a16z/dory`, 2022.

[2] A. Kate, G. M. Zaverucha, and I. Goldberg, "Constant-size commitments to polynomials and their applications," in *ASIACRYPT*, pp. 177–194, 2010.

[3] C. Papamanthou, E. Shi, and R. Tamassia, "Signatures of correct computation," in *TCC*, 2013.

[4] B. Chen, B. Bünz, D. Boneh, and Z. Zhang, "HyperPlonk: Plonk with linear-time prover and high-degree custom gates," in *EUROCRYPT*, 2023.

[5] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell, "Bulletproofs: Short proofs for confidential transactions and more," in *IEEE S&P*, 2018.

[6] J. Lee, "Dory: Efficient, transparent arguments for generalised inner products and polynomial commitments," in *TCC*, 2021.

[7] arkworks contributors, "`arkworks` zksnark ecosystem," 2022.